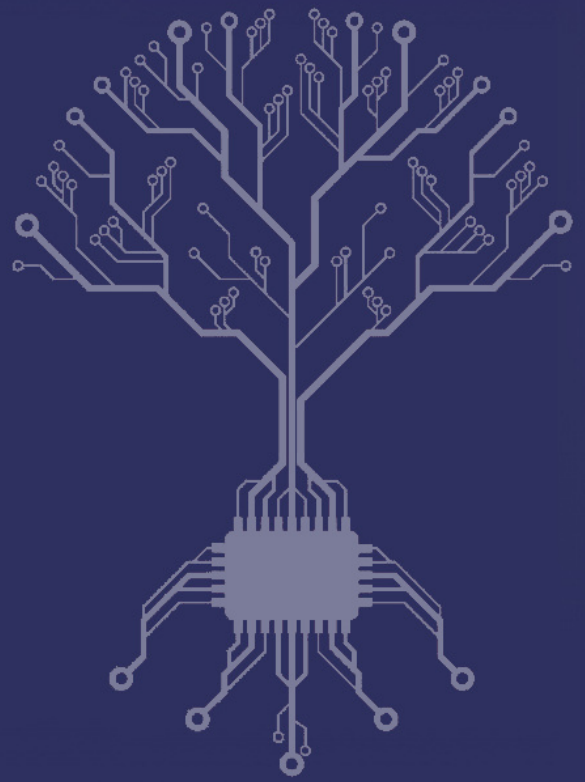
Base de Données (2/2)

Interrogation (Suite)



Jointure

Joindre deux tables, qui sont en relation à l'aide d'une clé primaire et d'une clé étrangère, revient à créer une nouvelle table temporaire qui associe les lignes qui ont des valeurs égales dans la colonne commune des deux tables.

Syntaxe

SELECT ... FROM Table1 JOIN Table2 ON Table1.colonne1 = Table2.colonne2 ;

Ou bien

SELECT ... FROM Table1 JOIN Table2 USING (colonne) ;

(si la colonne commune porte le même nom dans les deux tables)

Exemple

Soit les deux tables :

- *student*(<u>*student_id*</u>, *first_name*, *last_name*, *field*, *average*, #*center_code*)
- *center*(<u>*center_code*</u>, *name*, *city*)

student_id	first_name	last_name	average	center_code
1	Ali	Hassan	17.5	CP101
2	Sarah	Mohammad	18.3	CP102
3	Omar	Yussuf	16.0	CP301
4	Laila	Ahmad	14.4	CP102



center_code	name	city
CP101	Moulay Youssef	Rabat
CP102	Omar Al Khayyam	Rabat
CP301	Mohammed VI	Kénitra



student_id	first_name	last_name	average	center_code	name	city
1	Ali	Hassan	17.5	CP101	Moulay Youssef	Rabat
2	Sarah	Mohammad	18.3	CP102	Omar Al Khayyam	Rabat
3	Omar	Yussuf	16.0	CP301	Mohammed VI	Kénitra
4	Laila	Ahmad	14.4	CP102	Omar Al Khayyam	Rabat

Exemple

- Pour afficher les moyennes des étudiants dont le centre est *'Omar Al Khayyam'*, une jointure est nécessaire car la moyenne de l'étudiant et le nom du centre se trouvent dans des tables différentes :
  - SELECT average FROM student JOIN center ON student.center_code = center.center_code WHERE name = *'Omar Al Khayyam'* ;
  - ou SELECT average FROM student JOIN center USING (center_code) WHERE name = *'Omar Al Khayyam'* ;

La notation *table.colonne* désigne la colonne d'une table précise. Elle évite toute ambiguïté lorsque deux tables ont une colonne de même nom (par exemple *student.center_code* et *center.center_code*).

Requête imbriquée

Une requête imbriquée est une sous-requête exécutée à l'intérieur d'une autre requête SQL. Elle remplace souvent une constante au sein d'une clause WHERE ou HAVING.

Syntaxe

SELECT ... FROM ... WHERE ... opérateur (SELECT ... FROM ...) ;

Opérateur peut être :

- Si la sous-requête renvoie un seul résultat :
  - Un opérateur simple : $=$, $<$, $>$, $!=$, $<=$, $>=$
- Si elle renvoie plusieurs résultats :
  - IN
    NOT IN
  - EXISTS (la ligne est gardée si la sous-requête renvoie au moins une ligne)
    NOT EXISTS (la ligne est gardée si la sous-requête ne renvoie rien)

    La sous-requête utilise en général les valeurs de la ligne en cours de la requête principale.

Exemple 1

- Afficher les étudiants dont la moyenne est supérieure ou égale à la moyenne générale de tous les étudiants :
  - SELECT * FROM student WHERE average >= (SELECT AVG(average) FROM student) ;

- La sous-requête est exécutée une seule fois et renvoie une seule valeur : la moyenne générale.
- La requête principale compare ensuite la moyenne de chaque étudiant à cette valeur et garde ceux dont la moyenne est supérieure ou égale.
- On ne peut pas écrire directement WHERE average >= AVG(average) : une fonction d'agrégation n'est pas autorisée dans WHERE.

Exemple 2

- Afficher l'étudiant qui a obtenu la meilleure moyenne :
  - SELECT * FROM student WHERE average = (SELECT MAX(average) FROM student) ;

- La sous-requête renvoie la meilleure moyenne.
- La requête principale garde l'étudiant dont la moyenne est égale à cette valeur.
- SELECT MAX(average) FROM student seule donne la moyenne, mais pas l'étudiant qui l'a obtenue : d'où la sous-requête.
- Si plusieurs étudiants ont la même meilleure moyenne, ils sont tous affichés.

Exemple 3

- Soit les tables :

  student(*student_id*, *first_name*, *last_name*, *field*, *average*, #*center_code*)

  book(*ISBN*, *title*, *author*)

  borrow(#*student_id*, #*ISBN*, *date*)

- Afficher les étudiants qui ont emprunté un livre.

Exemple 3 (avec IN)

- SELECT student_id, first_name, last_name FROM student WHERE student_id IN (SELECT DISTINCT student_id FROM borrow) ;

- La sous-requête est exécutée une seule fois et renvoie la liste, sans doublons, des *student_id* présents dans la table *borrow*.
- La requête principale garde les étudiants dont le *student_id* appartient (IN) à cette liste.

Exemple 3 (avec EXISTS)

- SELECT student_id, first_name, last_name FROM student WHERE EXISTS (SELECT * FROM borrow WHERE student.student_id = borrow.student_id) ;

- Pour chaque étudiant de la requête principale, la sous-requête cherche dans *borrow* les emprunts ayant le même *student_id*.
- L'étudiant est gardé si la sous-requête renvoie au moins une ligne.